

Authentication Backend Contract

API requirements for candidate and recruiter registration, login, email verification, forgot password, and reset password.

Frontend ready for API integration

Current frontend status: The auth screens call the endpoints below. The single-document recruiter backend handoff is [Recruiter Backend API and Flow Handoff](#). For recruiter-only registration, login, email verification, and organization approval details, see the [Recruiter Login Flow Handoff](#). Recruiter workspace routes are in the separate [Recruiter API Handoff](#).

Project: MedHire

Document: Backend API handoff

Date: September 8, 2026

1. Common API Rules

Item	Requirement
Content type	Requests and responses use <code>application/json</code> .
Authentication	Return an access token after successful login. Prefer an HTTP-only secure refresh-token cookie or a secure token strategy.
Roles	Allowed values are <code>candidate</code> and <code>recruiter</code> .
Password	Minimum 8 characters. Recommended: at least one uppercase letter, number, and special character.
Verification code	Six numeric digits. Expire after approximately 10 minutes, allow resend rate limiting, and invalidate after successful use.
Reset token	Single-use, short-lived, and invalidated after a successful password reset.
Security	Hash passwords server-side. Never return passwords or expose whether an email exists during forgot-password requests.

2. Candidate Registration

POST `/api/auth/register`

```
{
  "role": "candidate",
  "fullName": "Anita Sharma",
  "email": "anita@example.com",
  "phone": "+919876543210",
  "password": "ExamplePassword@123",
  "termsAccepted": true
}
```

Frontend-only: `confirmPassword` is checked in the browser and is not sent to the backend. The terms checkbox becomes `termsAccepted`.

Success: Create the account in an unverified state, send a registration verification code, and return `201 Created`.

```
{
  "message": "Verification code sent",
  "userId": "user_123",
  "email": "anita@example.com",
  "role": "candidate",
  "emailVerified": false
}
```

3. Recruiter Registration

POST /api/auth/register

```
{
  "role": "recruiter",
  "fullName": "Rahul Menon",
  "email": "rahul@hospital.com",
  "phone": "+919876543210",
  "designation": "HR Manager",
  "organizationName": "MedCare Hospital",
  "organizationWebsite": "https://hospital.com",
  "city": "Chennai",
  "state": "Tamil Nadu",
  "password": "ExamplePassword@123",
  "termsAccepted": true
}
```

Frontend-only: `confirmPassword` is not sent. The UI currently displays the phone country prefix as `+91`; send the complete E.164 phone number or confirm a separate country-code field with the frontend team.

Success: Create the recruiter account as pending organization verification, send the email verification code, and return **201 Created**.

4. Login

POST /api/auth/login

```
{
  "email": "anita@example.com",
  "password": "ExamplePassword@123",
  "rememberMe": true
}
```

`rememberMe` controls token/session duration. It does not need to be stored as a user profile field.

Success: Return **200 OK** with session tokens and the authenticated user.

```
{
  "accessToken": "eyJ...",
  "refreshToken": "refresh-token-value",
  "expiresIn": 3600,
  "user": {
    "id": "user_123",
    "role": "candidate",
    "fullName": "Anita Sharma",
    "email": "anita@example.com",
    "emailVerified": true
  }
}
```

For recruiter accounts, include `organizationVerificationStatus` with one of `pending`, `approved`, or `rejected`. The frontend only grants recruiter dashboard access for `approved`.

Recruiter pending-review details: The frontend verification-pending page displays organization name, recruiter contact number, and registration date from the authenticated `user` object returned by this login

endpoint. Read these values from the existing recruiter/organization records; no separate pending-details endpoint is required.

```
{
  "accessToken": "eyJ...",
  "refreshToken": "refresh-token-value",
  "expiresIn": 3600,
  "user": {
    "id": "user_123",
    "role": "recruiter",
    "fullName": "Anita Sharma",
    "email": "anita@example.com",
    "emailVerified": true,
    "organizationId": "org_456",
    "organizationName": "MedCare Hospital",
    "phone": "+919876543210",
    "createdAt": "2026-09-24T10:30:00Z",
    "organizationVerificationStatus": "pending"
  }
}
```

`createdAt` is the recruiter's registration timestamp in ISO-8601 UTC. Keep the property names shown (`organizationName`, `phone`, `createdAt`) or coordinate a frontend mapping if the API uses different names. Return these fields for all recruiter statuses so the same session shape is consistent.

5. Forgot Password

POST /api/auth/forgot-password

```
{
  "email": "anita@example.com"
}
```

Success: Always return a generic response such as **202 Accepted**, whether or not the email exists.

```
{
  "message": "If an account exists, reset instructions have been sent."
}
```

6. Resend Verification Code

POST /api/auth/resend-code

```
{
  "email": "anita@example.com",
  "purpose": "registration"
}
```

For password reset, use "purpose": "password_reset". Rate-limit this endpoint, for example once per 60 seconds per email and IP address.

7. Verify Email or Reset Code

POST /api/auth/verify-code

```
{
  "email": "anita@example.com",
  "code": "482916",
  "purpose": "registration"
}
```

For registration, mark the account email as verified and return:

```
{
  "verified": true,
  "purpose": "registration",
  "user": {
    "id": "user_123",
    "role": "candidate",
    "emailVerified": true
  }
}
```

For password reset, use "purpose": "password_reset" and return a short-lived reset token:

```
{
  "verified": true,
  "purpose": "password_reset",
  "resetToken": "reset-token-value"
}
```

8. Reset Password

POST /api/auth/reset-password

```
{
  "email": "anita@example.com",
  "resetToken": "reset-token-value",
  "newPassword": "NewExamplePassword@123"
}
```

Frontend-only: `confirmPassword` is compared with `newPassword` in the browser and is not sent.

Success: Invalidate the token, update the password hash, revoke existing sessions if required, and return **200 OK**.

```
{
  "message": "Password reset successfully"
}
```

9. Error Response Format

```
{
  "error": {
```

```
"code": "VALIDATION_ERROR",
"message": "Please correct the highlighted fields.",
"fields": {
  "email": "Enter a valid email address."
}
}
```

Status	Use
400	Invalid request or validation failure.
401	Invalid login credentials or invalid reset token.
403	Email not verified or recruiter organization not approved.
409	Email already registered.
422	Business-rule validation failure.
429	Too many login, reset, or resend-code attempts.

10. Frontend Integration Checklist

- Connect candidate and recruiter registration forms to `POST /api/auth/register`.
- Connect login and store the returned session securely.
- Connect forgot-password form to `POST /api/auth/forgot-password`.
- Replace the demo verification code `123456` with the backend verification endpoint.
- Connect resend-code button to `POST /api/auth/resend-code`.
- Pass the returned reset token into the reset-password flow.
- Connect reset-password form to `POST /api/auth/reset-password`.
- Display backend validation and authentication errors in the form.
- Confirm whether the backend uses six-digit OTP verification or emailed reset links.
- Confirm the final API base URL and authentication token storage policy.

11. Current Frontend Files

Screen	Frontend file	Route
Candidate registration	src/pages/RegisterCandidate.jsx	/register/candidate
Recruiter registration	src/pages/RegisterRecruiter.jsx	/register/recruiter
Login	src/pages/Login.jsx	/login
Forgot password	src/pages/ForgotPassword.jsx	/forgot-password
Email verification	src/pages/EmailVerification.jsx	/verify-email
Reset password	src/pages/ResetPassword.jsx	/reset-password

Prepared for the MedHire backend team. This document describes the proposed API contract; backend endpoint names and token strategy should be confirmed before frontend integration.